CECS 229 – Assignment 6

Complex Numbers

Given the file, 'pa6.py', complete the code for the following functions:

1. `translate(S, z0)` – this function returns a set of complex numbers that were translated by z0.
    a. S – a set of complex numbers that will be translated by z0.
    b. z0 – the complex number to translate by.
    c. The returned value should be a set of complex numbers that were translated by z0.
    d. For each of the complex numbers in set S, add the complex number z0 to it and add the result to a new set. Then return the completed set.
2. `scale(S, k)` – this function returns a set of complex numbers that were scaled by factor k.
    a. S – a set of complex numbers that will be scaled by factor k.
    b. k – a positive float scale factor.
    c. The returned value should be a set of complex numbers.
    d. Raise a ValueError exception if k is negative or zero.
    e. For each of the complex numbers in set S, multiply the scale factor k to it and add the result to a new set. Then return the completed set.
3. `rotate(S, tau)` – this function returns a set of complex numbers that were rotated by tau radians.
    a. S – a set of complex numbers that will be rotated by tau.
    b. tau – a float rotation factor.
    c. The returned value should be a set of complex numbers that were rotated by tau.
    d. For each of the complex numbers in set S, apply the rotation factor tau to it and add the result to a new set. Then return the completed set.
    e. If tau is negative, then the rotation is clockwise. If tau is positive, then the rotation is counterclockwise. If tau is 0, then there is no rotation.
    f. You can use math.e in your calculation for the constant value e.

Complex Numbers in Python:

Python has built-in support for complex numbers as a standard numeric type. It uses the format: a + bj, where a is the real part, b is the imaginary part, and j is the imaginary unit (sqrt(-1)).

You can create a complex number in Python by either writing the complex number in the standard format, or by calling the constructor for a complex number.

Examples:

```
z1 = 2 + 3j
z2 = -5 + 0.5j
z3 = 1j  # equivalent to just i
z4 = complex(2, 3)
z5 = complex("2+3j")
```

You can access the real or imaginary part by using the properties of a complex number object.

Examples:
```
z = 4 + 3j
print(z.real) # 4.0
print(z.imag) # 3.0
```

Complex numbers override the arithmetic operators, such as, +, -, *, and /.

Examples:
```
z1 = 2 + 3j
z2 = 4 + 5j
print(z1 + z2)  # 6 + 8j
print(z1 - z2)  # -2 - 2j
print(z1 * z2)  # -7 + 22j
print(z1 / z2)  # 0.5609756097560976 + 0.0487804878048781j
```

Example Input/Output:
```
S = {2 + 2j, 3 + 3j, 4 + 4j}
translate(S, -3 - 2j)
{1j, (-1+0j), (1+2j)}
scale(S, 3)
{(6+6j), (12+12j), (9+9j)}
rotate(S, math.pi)
{(-4-4j), (-2-2j), (-3-3j)} #note: there may be rounding error
```

Notes:

1. Please place your name in the comment block at the top of the provided file.
2. Fill in the necessary code to complete the functions. Write your code where specified (ie. the TODOs). Do not modify the file name, function names, or parameters otherwise the program and tests will not work. Return the results in the format specified (ie. correct types and container types), otherwise the program and tests will not work properly.
3. Your solutions may not use any libraries or functions that perform the equivalent task.
4. Test your code using the provided 'main.py' file. You can write additional tests in the 'your_tester.py' file to further check your solutions. Note: just because the main tests pass does not guarantee that it will pass the tests on Gradescope.
5. Remember that sets have unique values and are unordered, this means that your results may not always have the order or number of results that you might initially expect.
6. When you are finished, submit your 'pa6.py' file to Gradescope on Canvas for further testing and grading. It should allow you to make repeated submissions.